Collaborative — Project Proposal

Oct 12, 2026 · Lucas Bixby

Problem and goal

Collaborative is a backend-focused sync platform that lets several people edit the same project in a creative application at the same time, starting with Blender. Google Docs does this for text; complex creative tools have no equivalent, so collaborators pass files back and forth.

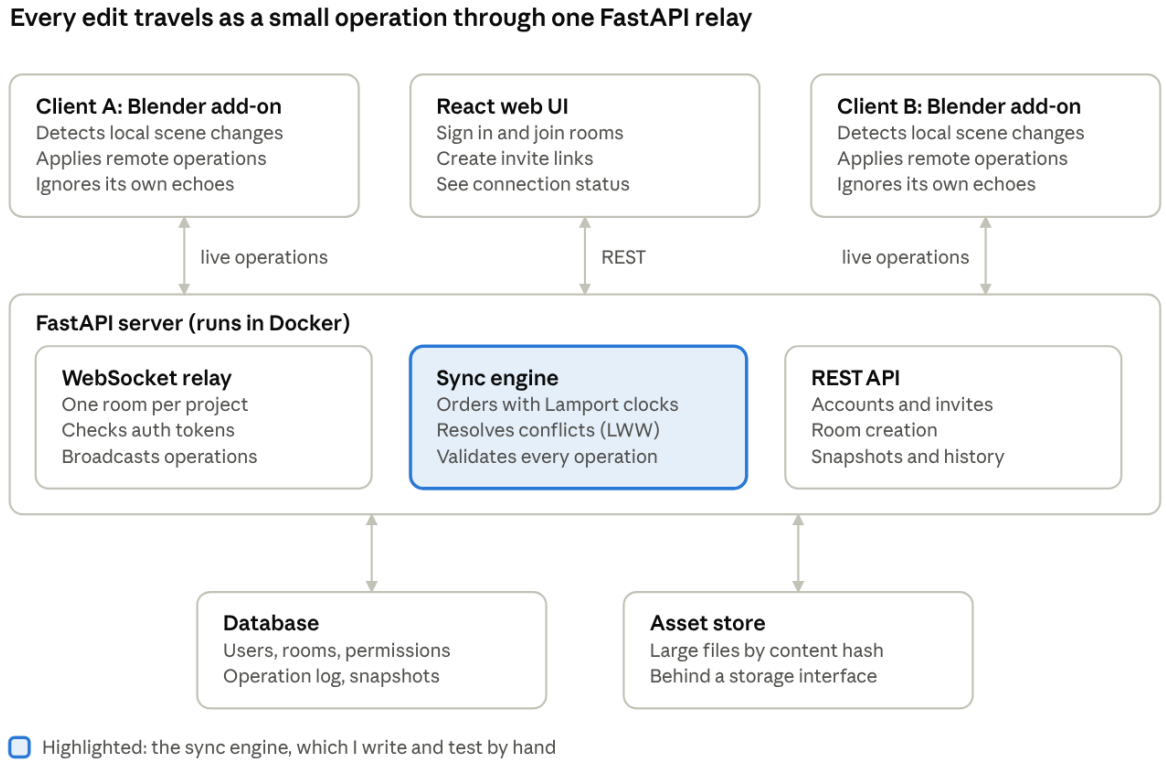
Goal: let two or more users each run their own copy of Blender, join a shared "room," and see each other's edits appear live. A background layer (a Blender add-on plus a relay server) captures changes, sends them through the backend, and applies them on every other client.

Legal note: every user still needs their own legal access to the application. Collaborative only syncs project changes; it does not host, bundle, or modify the application itself.

Why this project: the work is mostly backend and architecture. It draws on data structures and algorithms (operation logs, conflict resolution), security and privacy (auth, room permissions, validating untrusted input), code optimization (compact deltas, low latency), and operating systems (concurrency, networking, processes).

Architecture: the sync model

Each user runs Blender locally with the add-on, and the server never runs Blender. Only small operations travel between users, so edits show up live.



sync architecture · 2 clients, 1 server, 2 stores

One FastAPI server sits between all clients, and everything it needs to remember lives in the database and the asset store.

How one change travels

1. The add-on detects a change in Blender and wraps it as an operation (object, property, new value, timestamp, origin ID).
2. It sends the operation over the WebSocket to the room.
3. The sync engine checks the sender's permissions, validates the operation, and orders it with a Lamport timestamp.
4. The server appends it to the operation log and broadcasts it to the other clients in the room.
5. Each remote add-on applies the operation and does not re-send it, because the origin ID marks it as a remote change.
6. A client that reconnects asks for the operations it missed, or loads the latest snapshot first.

Tech stack

FastAPI is the backend for everything, including real-time sync, storage, and auth.

Layer	Choice	Role
Client integration	Blender add-on (Python)	Detects local scene changes, sends them as operations, applies remote operations
Relay server	FastAPI + WebSockets	Rooms, auth, ordering and broadcasting of operations
REST API	FastAPI	Accounts, room creation and invites, project snapshots and history
Database	SQLite for development, Postgres if time allows	Users, rooms, operation log, snapshots
Asset storage	Local disk behind a storage interface	Large files (meshes, textures) stored by content hash
Deployment	Docker	Packages and runs the server the same way everywhere (the client apps stay on users' machines)
Web UI	React	Small app for sign-in, creating and joining rooms, connection status
Dev workflow	Claude Code (optionally with Ponytail)	Boilerplate, debugging, review passes

Decision: FastAPI over Cloudflare. One server I control makes the sync logic easier to build, test, and reason about than splitting state between FastAPI and Cloudflare Workers or Durable Objects. Putting storage behind an interface means object storage such as Cloudflare R2 can be swapped in later without touching the sync engine.

AI tooling rule: use Claude Code freely for scaffolding and the UI. Hand-write and fully understand the sync engine (operation format, ordering, conflict handling), and cover it with tests.

Features and scope

The committed scope is Blender only, syncing scene changes between users in a shared room. Other applications are future adapters, not part of this project; Adobe's extension systems are limited and bound by their terms of service.

In scope (must have)

- Blender add-on that detects object changes and applies remote ones without echo loops
- Sync of object transforms (location, rotation, scale)
- Object creation and deletion
- FastAPI WebSocket relay with rooms, auth tokens, and per-room permissions
- Operation log with ordering (Lamport timestamps) and last-writer-wins conflict handling
- Persistence: room snapshots and operation history, with catch-up after a client reconnects
- Dockerized server and a minimal React UI for rooms and status

Stretch

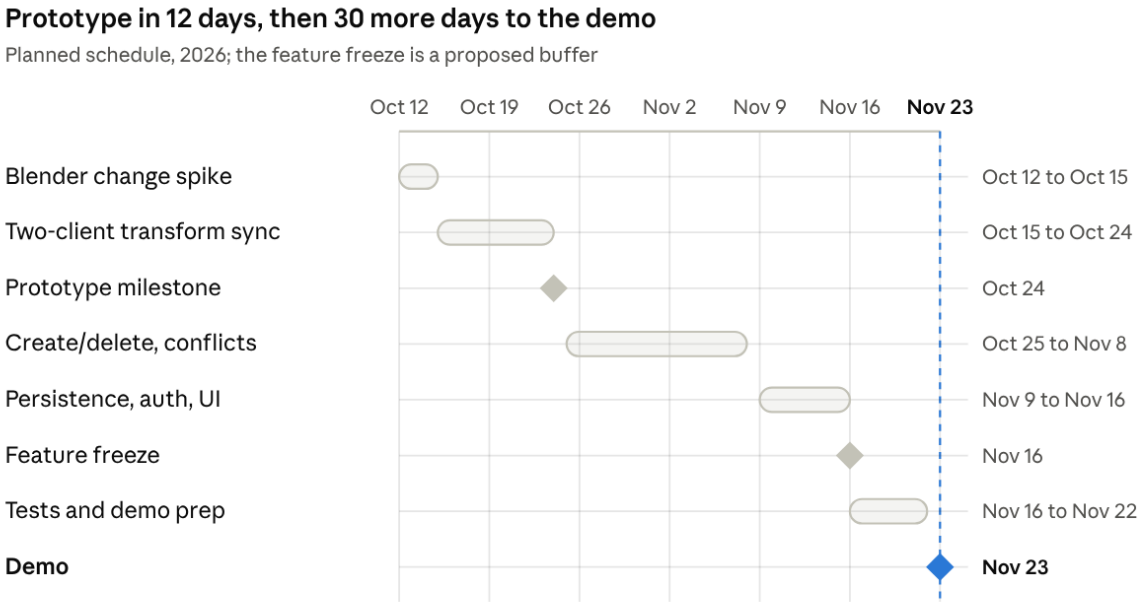
- Simple mesh edits (vertex moves) with compact deltas
- Content-addressed asset storage for textures and large meshes
- Presence (who is in the room, whose selection is whose)
- More careful conflict handling (CRDT-style merging for specific properties)

Out of scope

- Adobe products or any other application
- Streaming or hosting the applications themselves
- Reverse engineering closed-source software

Timeline and milestones

The working prototype is due on 10/24 and the demo on 11/23.



schedule · 10/12 to 11/23, to scale

Prototype milestone (10/24): two Blender instances syncing object transforms through the FastAPI relay. I propose freezing features on 11/16, a week before the demo, so the last week goes to tests, measurements and demo preparation.

Success criteria

The project succeeds if two Blender instances stay in the same state while both users edit, and the results can be measured.

- ☐ Two users join the same room and see each other's object transforms update live
- ☐ Object creation and deletion sync both ways
- ☐ Both clients converge to identical scene state after simultaneous edits (verified by automated tests with simulated concurrent clients)
- ☐ Sync latency is under 200 ms on a local network (target to confirm once measured)
- ☐ A client that disconnects and reconnects recovers the current state
- ☐ Room snapshots and operation history persist across a server restart
- ☐ Only authenticated, permitted users can join a room or send operations
- ☐ Server runs from a single Docker command

Risks and mitigations

The biggest risk is that the Blender change-detection spike doesn't work as hoped, so it comes first.

Risk	Likelihood	Mitigation
Blender's Python API can't reliably report some changes	Medium	Spike in days 1-3; fall back to polling scene state at a short interval for the properties I sync
Echo loops (applying a remote change triggers a new outgoing change)	High	Tag operations with an origin ID and ignore changes caused by applying remote operations
Concurrent edits to the same property diverge	Medium	Lamport timestamps and last-writer-wins first; add smarter merging only if time allows
Large mesh and texture data is slow to sync	Medium	Send small deltas; keep assets separate and reference them by hash; mesh sync is a stretch goal
Security: malicious or malformed operations from a client	Medium	Authenticate sockets, validate operation schema and size, enforce room permissions server-side
Time: 12 days to prototype, about 6 weeks total	High	Fixed scope, features frozen about a week before the demo, stretch goals only after must-haves work

Success criteria

The project succeeds if two Blender instances stay in the same state while both users edit, and the results can be measured.

- □ Two users join the same room and see each other's object transforms update live
- □ Object creation and deletion sync both ways
- □ Both clients converge to identical scene state after simultaneous edits (verified by automated tests with simulated concurrent clients)
- □ Sync latency is under 200 ms on a local network (target to confirm once measured)
- □ A client that disconnects and reconnects recovers the current state
- □ Room snapshots and operation history persist across a server restart
- □ Only authenticated, permitted users can join a room or send operations
- □ Server runs from a single Docker command